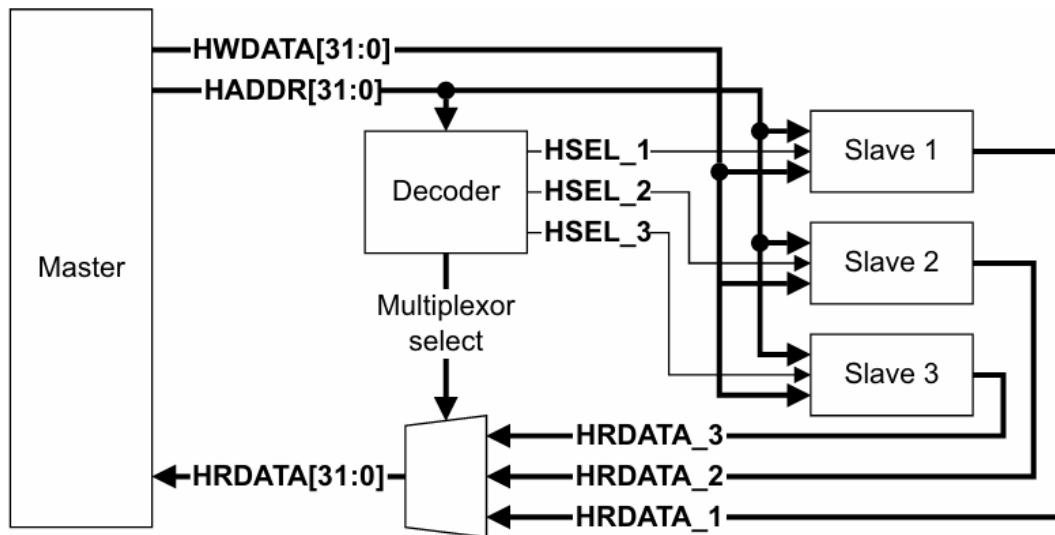




Summer 2025' Digital Communication and  
Digital Design Internship

# AMBA<sup>®</sup>3 AHB-Lite Protocol v1.0



**By: Khaled Ahmed Hamed**

For Github Repo: [Click Here](#)

## Contents

|                                                 |    |
|-------------------------------------------------|----|
| AHB-Lite Master-Slave Verification Report ..... | 3  |
| 1. Introduction .....                           | 3  |
| 2. AHB-Lite Architecture .....                  | 4  |
| Global Signals (HCLK & HRESETn).....            | 4  |
| 2.1 Master .....                                | 5  |
| Master's operational FSM: .....                 | 6  |
| 2.2 Slave .....                                 | 7  |
| 2.3 Interconnection .....                       | 8  |
| 2.3.1 Decoder .....                             | 8  |
| 2.3.2 Multiplexor .....                         | 8  |
| 3. Verilog HDL Design Description .....         | 9  |
| 3.1 AHB-Lite Slave.....                         | 9  |
| 3.2 AHB-Lite Master.....                        | 10 |
| 3.3 AHB-Lite Interconnection.....               | 11 |
| 4. Testbench and Simulation .....               | 12 |
| 4.1 Simulation Environment .....                | 12 |
| 4.2 Directed Test Cases .....                   | 13 |
| 5. Future Work .....                            | 15 |
| 6. Conclusion.....                              | 15 |

## Figures & Tables

|                                                                  |    |
|------------------------------------------------------------------|----|
| Figure 1: AHB-Lite block diagram .....                           | 4  |
| Figure 2: Master interface .....                                 | 5  |
| Figure 3: Master's FSM Operation.....                            | 6  |
| Figure 4: Slave interface .....                                  | 7  |
| Figure 5: Slave select signals.....                              | 8  |
| Figure 6: Multiplexor interconnection .....                      | 8  |
| Figure 7: Write Operation.....                                   | 12 |
| Figure 8: Read Operation.....                                    | 12 |
| Figure 9: Master's Write Operation .....                         | 12 |
| Figure 10: Master's Read Operaion .....                          | 12 |
| Figure 11: Master's Operation Transcript .....                   | 13 |
| Figure 12: Slave's Write and Read Operations.....                | 13 |
| Figure 13: Slave's Operation Transcript .....                    | 13 |
| Figure 14: Multiple Transfers .....                              | 14 |
| Figure 15: Whole System Multiple Write and Read Operations ..... | 14 |
| Figure 16: Whole System Transcript .....                         | 14 |

# AHB-Lite Master-Slave Verification Report

## 1. Introduction

The objective of this project is to design, implement, and verify **AHB-Lite Master-Slave System**.

AHB-Lite is a subset of the AMBA AHB (Advanced High-performance Bus) designed for systems with a single master and multiple slaves. It supports **high-performance, pipelined, burst-based, and single-cycle** transfers. This system includes:

- A single configurable AHB-Lite Master
  - Three AHB-Lite compliant memory-mapped Slaves
  - Address Decoder and Read Multiplexor
  - 3 Verilog Testbenches for directed simulation for Master, Slave and the Whole SoC
-

## 2. AHB-Lite Architecture

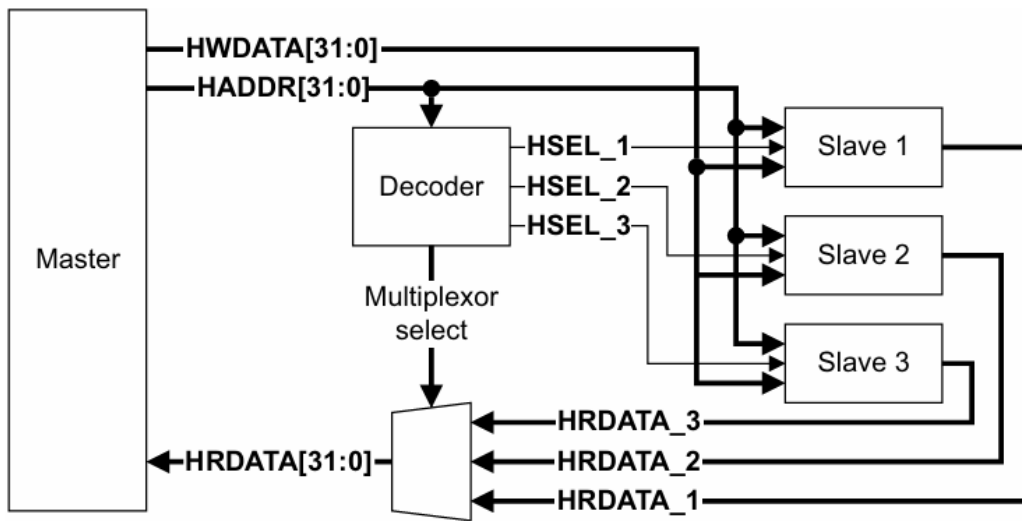


Figure 1: AHB-Lite block diagram

The AMBA AHB-Lite system implemented here consists of a single AHB master interfaced with multiple AHB-compliant slaves via an AHB interconnection module. This architecture allows the master to perform read and write operations to different memory-mapped regions handled by each slave.

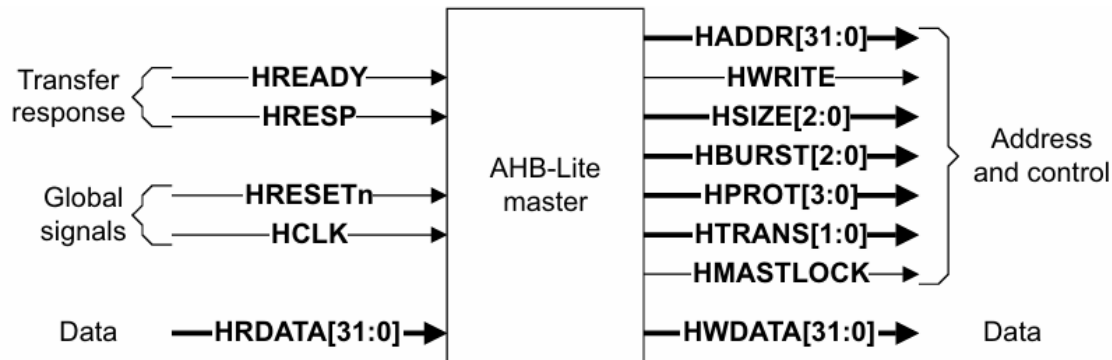
The master initiates transactions using control signals such as (**HTRANS**, **HWRITE**, and **HADDR**), while each slave responds based on address decoding and readiness signals (**HREADYOUT**, **HRESP**).

The interconnect routes transactions to the correct slave based on address range mapping. The design ensures single-cycle address and control transfer, followed by one or more cycles of data transfer, aligning with the AHB-Lite protocol specified in ARM IHI0033A. The entire system supports protocol-compliant handshaking and error handling.

### Global Signals (HCLK & HRESETn)

- **HCLK**: Each AHB-Lite component uses a single clock signal, HCLK. All input signals are sampled on the rising edge of **HCLK**. All output signal changes must occur after the rising edge of HCLK.
- **HRESETn**: The reset signal, **HRESETn**, is the only active LOW signal in the AHB-Lite protocol. The reset can be asserted asynchronously, but is deasserted synchronously after the rising edge of **HCLK**. During reset all masters must ensure the address and control signals are at valid levels and that **HTRANS[1:0]** indicates IDLE and all slaves must ensure that **HREADYOUT** is HIGH

## 2.1 Master



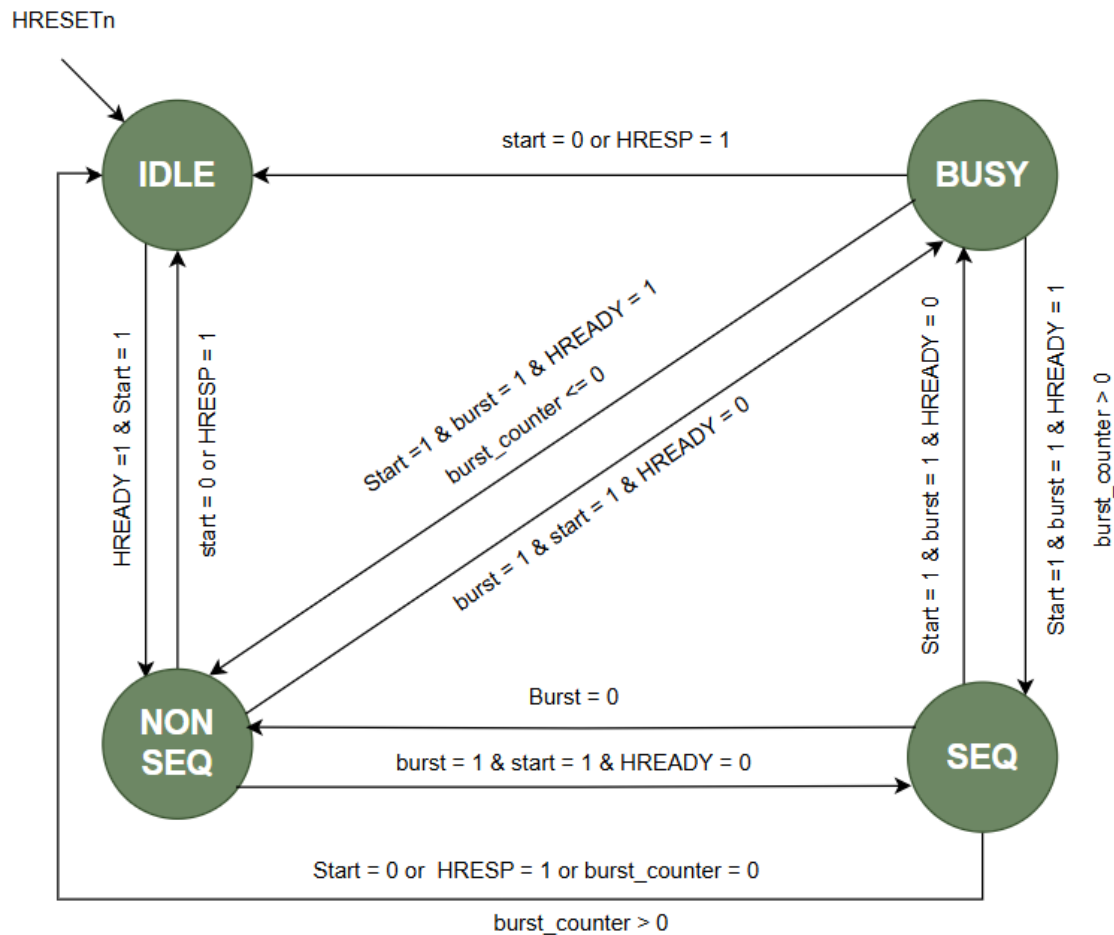
*Figure 2: Master interface*

The AHB-Lite master initiates all read and write transfers by generating address, control, and data signals. It supports:

- **HTRANS:** transfer type (IDLE, NONSEQ, SEQ, BUSY)
- **HWRITE:** read/write direction
- **HADDR, HSIZE, HBURST:** address, size (byte/halfword/word), burst type
- **HWDATA:** write data
- **HRDATA:** read data returned from slave
- **HREADY:** wait state control
- **HRESP:** response signaling from slave

The master begins a transaction by driving a **NONSEQ** transfer on the bus, indicating the start of a new access. During write operations, it provides the address and control information in one cycle, followed by the data in the next cycle. For read operations, the master drives the address and waits for the **HRDATA** response from the slave. It monitors the **HREADY** and **HRESP** signals to determine whether the slave is ready or an error has occurred. The master ensures that pipelined and burst operations are supported as per the protocol, with synchronization and arbitration handled internally since AHB-Lite supports only a single master.

## Master's operational FSM:

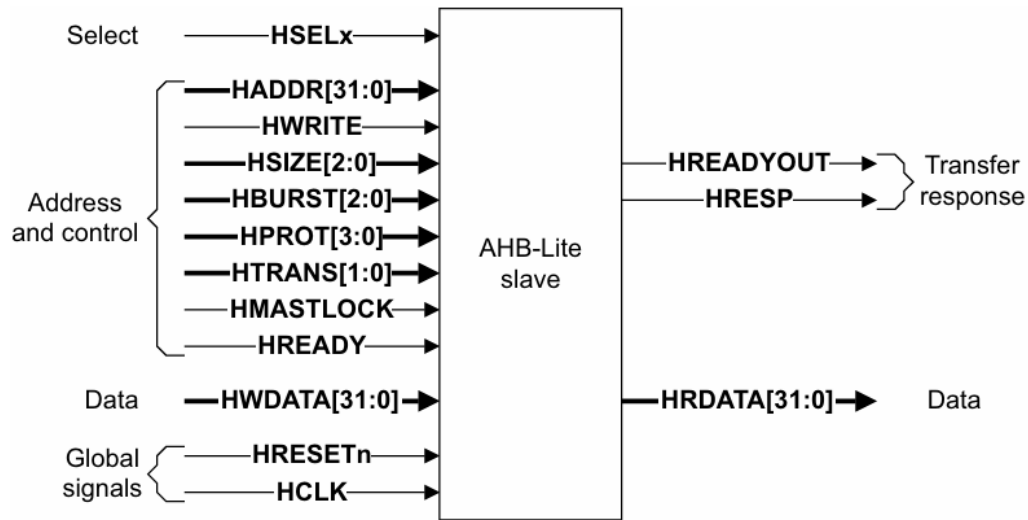


**Figure 3: Master's FSM Operation**

The master FSM governs the sequence of transactions on the AHB-Lite bus. It consists of four states: **IDLE**, **NONSEQ**, **SEQ**, and **BUSY**.

Initially, the FSM remains in the **IDLE** state until a valid transaction is initiated (**start** = 1 and **HRESP** = 0). Upon start, the FSM transitions to the **NONSEQ** state to begin a new transfer with a new address. If a burst transfer is enabled (**burst** = 1), and **HREADY** is high, the FSM moves to the **SEQ** state for subsequent sequential transfers. If the bus is not ready (**HREADY** = 0), it enters the **BUSY** state, waiting until the slave responds. Once all data beats are transferred (**burst\_counter** = 0), or if **start** = 0 or an error occurs (**HRESP** = 1), the FSM returns to the **IDLE** state. This FSM Implementation ensures compliance with AHB-Lite timing and handshake protocol, enabling efficient pipelined and burst transactions.

## 2.2 Slave



*Figure 4: Slave interface*

Each AHB-Lite slave responds to transfers from the master. Key signals include:

- **HREADYOUT**: indicates transfer complete
- **HRESP**: OKAY/ERROR response
- **HRDATA**: read data
- **HSELx**: select signal from decoder

Each AHB-Lite slave monitors the bus for address ranges assigned to it and responds to the master's requests if the **HADDR** matches its base address and a valid transfer is indicated by **HTRANS[1] = 1**. Upon a valid read or write request, the slave checks for alignment, size, and range errors, and either accepts the transfer or signals an error via **HRESP**. During write operations, the slave latches incoming data from **HWDATA** and stores it in internal memory. For read operations, the slave provides the appropriate data through **HRDATA** while setting **HREADYOUT** high to indicate completion. The slave strictly follows the AHB-Lite handshaking protocol, ensuring correct timing of data and control signals, and supports byte, halfword, and word accesses as required by the master.

## 2.3 Interconnection

### 2.3.1 Decoder

The **decoder** selects one slave at a time based on the upper bits of the address

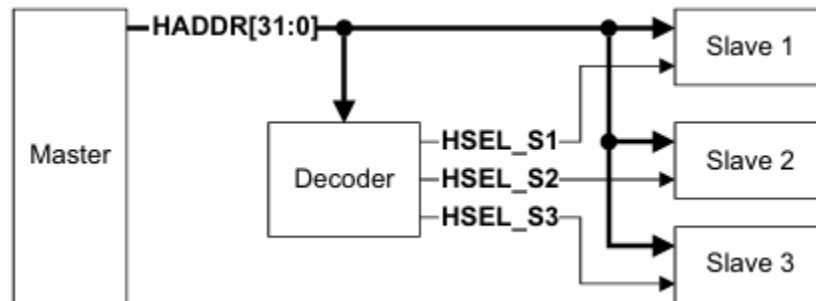


Figure 5: Slave select signals

### 2.3.2 Multiplexor

The **multiplexor** returns HRDATA, HRESP, and HREADYOUT from the selected slave back to the master.

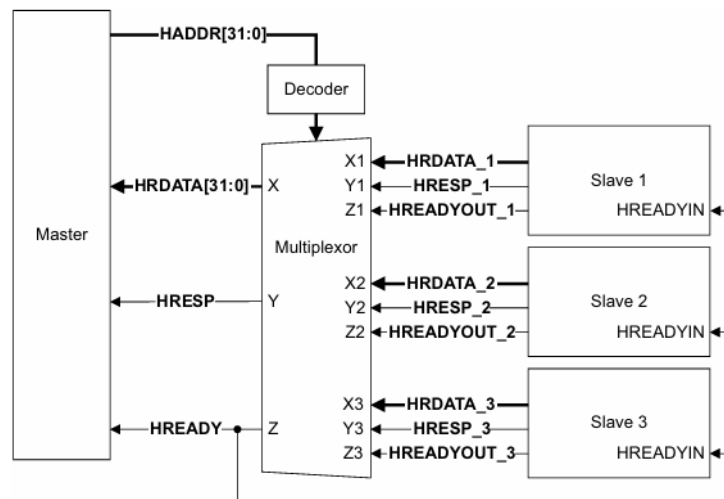


Figure 6: Multiplexor interconnection

## 3. Verilog HDL Design Description

This section details the RTL implementation of the AHB-Lite system components using Verilog HDL. The design includes a single master, three slaves, and a centralized interconnect module that routes transactions based on address decoding logic. All modules comply with the AMBA AHB-Lite protocol (ARM IHI0033A), maintaining proper handshaking, pipelining, and timing rules.

---

### 3.1 AHB-Lite Slave

Each AHB-Lite slave implements a dedicated **1 KB local memory region**, with support for **byte**, **halfword**, and **word** accesses. The slave monitors address alignment and transfer size, rejecting invalid accesses using **HRESP** and **HREADYOUT** signaling.

#### Functional Highlights

- **Memory Mapping:** Each slave is mapped to a distinct 1KB region using **BASE\_ADDR**.
- **Access Types Supported:**
  - Byte (8-bit)
  - Halfword (16-bit)
  - Word (32-bit)
- **Pipelined Transfers:** Input control signals are latched only when **HREADYIN** = 1.
- **Error Checking:**
  - **Misaligned address access**
  - **Unsupported HSIZE values**
  - **Out-of-range address violations**

#### Address alignment follows the ARM requirement:

- Word access: aligned to 4 bytes
  - Halfword access: aligned to 2 bytes
- Misalignment triggers an error response via **HRESP** = 1.

#### Timing Behavior

The slave introduces no wait states under normal operation. However, on error conditions, it uses a 2-cycle response via **HREADYOUT** to signal error and recovery, in accordance with AHB-Lite protocol.

## 3.2 AHB-Lite Master

The AHB-Lite master is responsible for generating **directed stimulus** to verify slave and interconnect functionality. It includes a fully featured **Finite State Machine (FSM)** and address generator capable of performing **burst and single** transfers.

### Supported Transfer Types

- **SINGLE Read/Write**
- **BURST Transfers: INCR4, INCR8, INCR16**
- **Wait-State Handling**
- **Edge-Case Testing:**
  - Illegal **HSIZE** values
  - Burst boundary violations
  - HRESP error recovery

### FSM Phases

1. **Address Phase:** Sets **HTRANS**, **HADDR**, **HWRITE**, **HSIZE**, and **HBURST**.
2. **Data Phase:** Drives **HWDATA** (for writes) or captures **HRDATA** (for reads).
3. **Wait for Slave:** Waits on **HREADY** = 1 before proceeding.
4. **Post-Transfer Logic:** Updates counters and FSM transitions.

The master FSM transitions among **IDLE**, **NONSEQ**, **SEQ**, and **BUSY**, coordinating burst and single transfers. Detailed FSM diagram is shown in Section 2.

### 3.3 AHB-Lite Interconnection

The **AHB-Lite Interconnection Module** is a central multiplexer-based router that connects one master to multiple slaves.

#### Functionality

- **Address Decoding:** Uses **HADDR[13:14]** to select the active slave.
- **Data Routing:** Forwards **HRDATA**, **HRESP**, and **HREADYOUT** from the selected slave to the master.
- **Handshake Multiplexing:**
  - Only the addressed slave receives the full transfer.
  - Non-selected slaves remain idle.

#### Address Mapping Table

| Slave ID | Base Address | Memory Size |
|----------|--------------|-------------|
| 0        | 0x4000 0000  | 1 KB        |
| 1        | 0x4000 2000  | 1 KB        |
| 2        | 0x4000 4000  | 1 KB        |

*Table 1: Slaves Addresses Mapping Table*

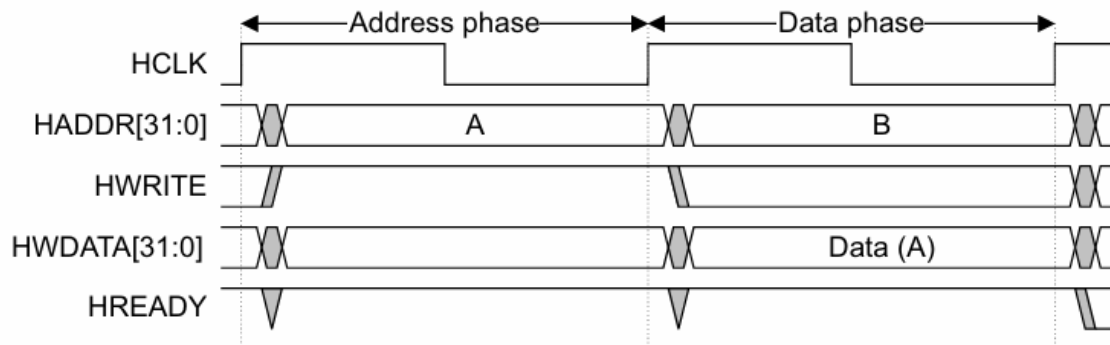
Addresses outside these ranges are **not routed** and may cause undefined behavior if not trapped.

## 4. Testbench and Simulation

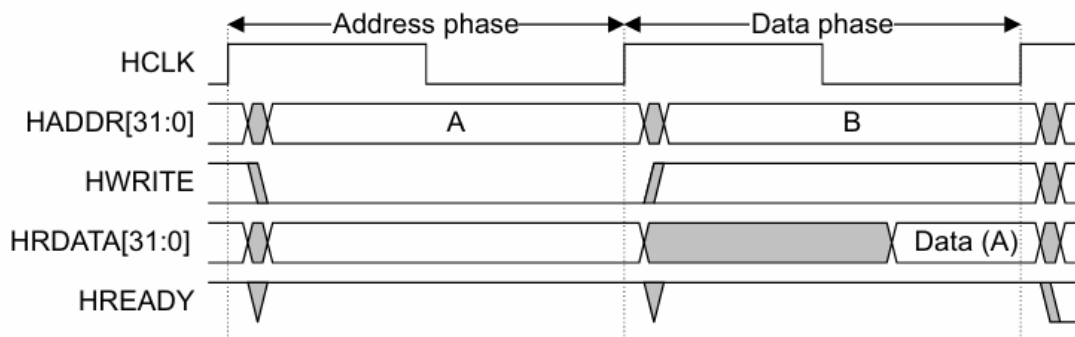
### 4.1 Simulation Environment

**Clock:** 10 ns period

**Reset:** Active-low async reset

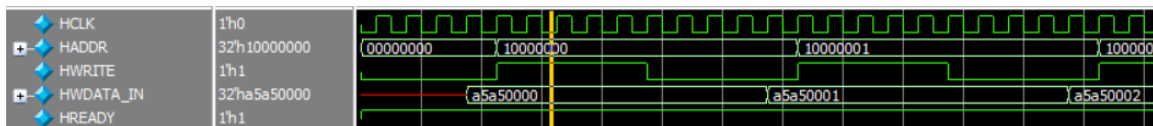


**Figure 7: Write Operation**

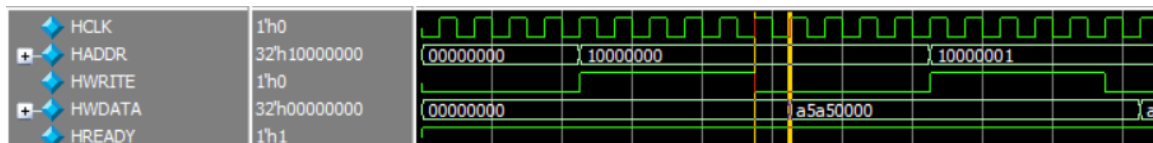


**Figure 8: Read Operation**

### Verifying Master:



**Figure 9: Master's Write Operation**



**Figure 10: Master's Read Operation**

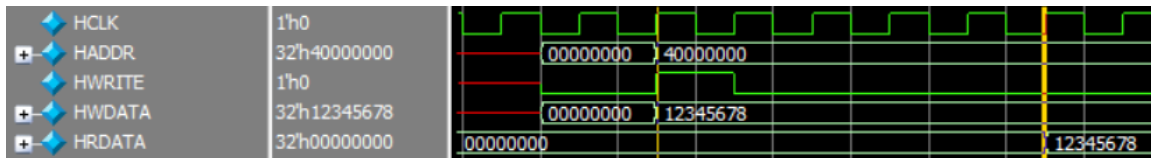
```

# ==== AHB-Lite Master Verification Testbench ====
# [35000 ns] TEST: SINGLE WRITE | HSIZE=0
# [85000 ns] TEST: SINGLE READ | HSIZE=0
# [135000 ns] TEST: SINGLE WRITE | HSIZE=1
# [185000 ns] TEST: SINGLE READ | HSIZE=1
# [235000 ns] TEST: SINGLE WRITE | HSIZE=2
# [285000 ns] TEST: SINGLE READ | HSIZE=2
# [335000 ns] TEST: BURST WRITE | HSIZE=1 | HBURST=0b001 (8 beats)
# [475000 ns] TEST: BURST READ | HSIZE=1 | HBURST=0b001 (8 beats)
# [615000 ns] TEST: BURST WRITE | HSIZE=2 | HBURST=0b011 (4 beats)
# [715000 ns] TEST: BURST READ | HSIZE=2 | HBURST=0b011 (4 beats)
# [815000 ns] TEST: BURST WRITE | HSIZE=2 | HBURST=0b101 (8 beats)
# [955000 ns] TEST: BURST READ | HSIZE=2 | HBURST=0b101 (8 beats)
# [1095000 ns] TEST: BURST WRITE | HSIZE=0 | HBURST=0b111 (16 beats)
# [1315000 ns] TEST: BURST WRITE | HSIZE=0 | HBURST=0b111 (16 beats)
# [1535000 ns] TEST: WAIT STATE
# [1615000 ns] TEST: ERROR RESPONSE

```

**Figure 11: Master's Operation Transcript**

## Verifying Slave:



**Figure 12: Slave's Write and Read Operations**

```

# Write: Addr = 0x40000000, Data = 0x12345678
# PASS : Addr = 0x40000000 | Data = 0x12345678
# Write: Addr = 0x40000004, Byte = 0xab
# PASS : Addr = 0x40000004 | Byte = 0xab
# Write: Addr = 0x40000008, Half = 0xcdef
# PASS : Addr = 0x40000008 | Half = 0xcdef
# Write: Addr = 0x4000000c, Data = 0xa5a5a5a5
# PASS : Addr = 0x4000000c | Data = 0xa5a5a5a5

```

**Figure 13: Slave's Operation Transcript**

## 4.2 Directed Test Cases

| Operation    | Address    | Expected Data | Result |
|--------------|------------|---------------|--------|
| Write SINGLE | 0x40000000 | 0x11223344    | PASS   |
| Read SINGLE  | 0x40000000 | 0x11223344    | PASS   |
| Write SINGLE | 0x40002000 | 0xdeadbeef    | PASS   |
| Read SINGLE  | 0x40002000 | 0xdeadbeef    | PASS   |
| Write SINGLE | 0x40004000 | 0xaabbccdd    | PASS   |
| Read SINGLE  | 0x40004000 | 0xaabbccdd    | PASS   |

**Table 2: SoC Test Cases**

See Figures 3-1 to 3-5 in the spec for read/write transfer timing diagrams.

## Verifying Whole System:

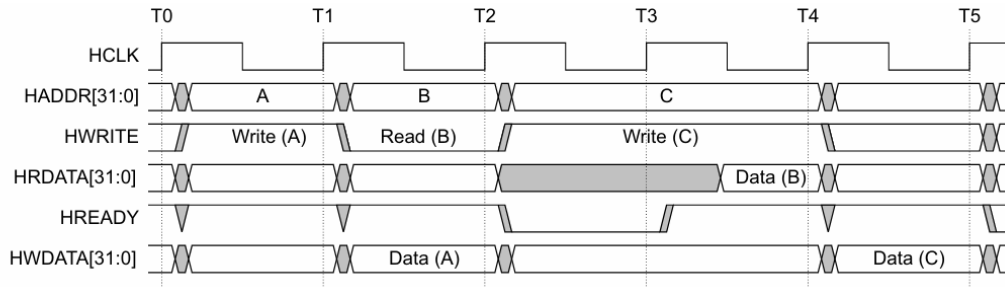


Figure 14: Multiple Transfers

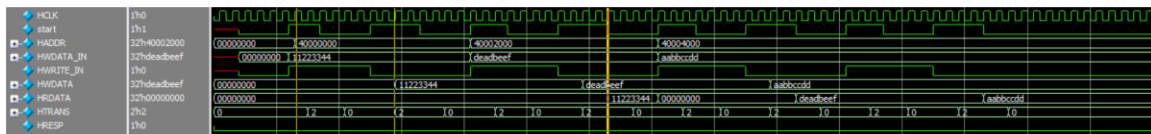


Figure 15: Whole System Multiple Write and Read Operations

```
# ==== Starting Master-Slave System Test ====
# Writing 0x11223344 to 0x40000000 60000
# [65000] HTRANS: 00, HADDR: 40000000, HWRITE: 1, HWDATA: 00000000, HRDATA: 00000000, HREADY: 1
# [75000] HTRANS: 10, HADDR: 40000000, HWRITE: 1, HWDATA: 00000000, HRDATA: 00000000, HREADY: 1
# Write complete at 85000
# [105000] HTRANS: 00, HADDR: 40000000, HWRITE: 1, HWDATA: 00000000, HRDATA: 00000000, HREADY: 1
# Reading from 0x40000000 at 125000
# [125000] HTRANS: 00, HADDR: 40000000, HWRITE: 0, HWDATA: 00000000, HRDATA: 00000000, HREADY: 1
# [145000] HTRANS: 10, HADDR: 40000000, HWRITE: 0, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# Read data: 0x11223344 at 165000
# [185000] HTRANS: 00, HADDR: 40000000, HWRITE: 0, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# Writing 0xdeadbeef to 0x40002000 205000
# [205000] HTRANS: 00, HADDR: 40002000, HWRITE: 1, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# [225000] HTRANS: 10, HADDR: 40002000, HWRITE: 1, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# Write complete at 235000
# [255000] HTRANS: 00, HADDR: 40002000, HWRITE: 1, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# Reading from 0x40002000 at 275000
# [275000] HTRANS: 00, HADDR: 40002000, HWRITE: 0, HWDATA: 11223344, HRDATA: 00000000, HREADY: 1
# [295000] HTRANS: 10, HADDR: 40002000, HWRITE: 0, HWDATA: deadbeef, HRDATA: 00000000, HREADY: 1
# Read data: 0xdeadbeef at 315000
# [315000] HTRANS: 10, HADDR: 40002000, HWRITE: 0, HWDATA: deadbeef, HRDATA: 11223344, HREADY: 1
# [335000] HTRANS: 00, HADDR: 40002000, HWRITE: 0, HWDATA: deadbeef, HRDATA: 11223344, HREADY: 1
# Writing 0xaabbccdd to 0x40004000 355000
# [355000] HTRANS: 00, HADDR: 40004000, HWRITE: 1, HWDATA: deadbeef, HRDATA: 00000000, HREADY: 1
# [375000] HTRANS: 10, HADDR: 40004000, HWRITE: 1, HWDATA: deadbeef, HRDATA: 00000000, HREADY: 1
# Write complete at 385000
# [405000] HTRANS: 00, HADDR: 40004000, HWRITE: 1, HWDATA: deadbeef, HRDATA: 00000000, HREADY: 1
# Reading from 0x40004000 at 425000
# [425000] HTRANS: 00, HADDR: 40004000, HWRITE: 0, HWDATA: deadbeef, HRDATA: 00000000, HREADY: 1
# [445000] HTRANS: 10, HADDR: 40004000, HWRITE: 0, HWDATA: aabbccdd, HRDATA: 00000000, HREADY: 1
# Read data: 0xaabbccdd at 465000
```

Figure 16: Whole System Transcript

- **HTRANS** toggles between **NONSEQ** and **SEQ** correctly in bursts
- **HWRITE** and **HWDATA** stable through valid transfers
- **HRDATA** matches expected memory contents
- **HRESP** = 1 correctly flags error on misaligned access

## 5. Future Work

- Upgrade to perform **Multi-Master** operation.
  - Implement **wrapping bursts (WRAP4/8/16)**, **locked transfers**, **Protection and Burst termination**.
  - Integrate **UVM Environment** for coverage-driven verification.
  - Add **Randomized Testing** with constrained scenarios.
  - Insert **Assertions and Formal Checks**.
  - **Run Backend flow to generate GDSII file**.
- 

## 6. Conclusion

This report documents a complete and robust implementation of an AMBA AHB-Lite system with one master and multiple memory-mapped slaves. The Verilog design adheres to ARM's specification and has been verified via exhaustive simulation. The system demonstrates key concepts of pipelined transfers, burst management, and modular decoding.

For more details on transfer types, burst handling, and slave responses, refer to:

[ARM IHI0033A: AMBA AHB-Lite Specification](#)